

Lista de Revisão — Preparação para a Avaliação 3

Turma: LionsDev **Tópicos:** MongoDB e Mongoose, autenticação com bcrypt e JWT, camadas do boilerplate, testes automatizados, Swagger, debug e deploy (Módulos 8, 9, 9B e 10).

Objetivo: Esta lista prepara para a Avaliação 3. Os exercícios usam uma API de filmes; a prova usa outro recurso, com os mesmos raciocínios. Os exercícios 3, 4, 6 e 7 podem ser feitos e testados **sem banco**, com mocks.

1. Schema com validação

Crie o model **Filme** com **titulo** (texto, obrigatório, mensagem "O título é obrigatório."), **ano** (número, entre 1888 e o ano atual), **generos** (lista de textos), **nota** (número de 0 a 10) e **excluidoEm** (data, padrão **null**), com **timestamps**. Mostre as mensagens de erro de um filme inválido usando **validateSync()**, que funciona sem conectar no banco.

2. Embedding ou referência?

Para cada caso, diga se usaria embedding ou referência e por quê: (a) os endereços de entrega de um cliente; (b) as avaliações de um filme, escritas por usuários; (c) o diretor de um filme, que também tem página própria.

3. Rota de busca com tratamento de erro

Escreva **GET /filmes/:id** com **async / await** e **try / catch**: 200 com o filme, **404** se não existir (ou se tiver **excluidoEm**), **400** se o id tiver formato inválido (**CastError**).

4. Atualização parcial e soft delete

- **PATCH /filmes/:id** altera só **titulo**, **ano**, **generos** e **nota**; qualquer outro campo do body é ignorado.
- **DELETE /filmes/:id** não apaga: grava a data em **excluidoEm** e responde 204.
- **GET /filmes** lista só os filmes com **excluidoEm: null**.

Dica: monte um objeto só com os campos permitidos antes de chamar o `findOneAndUpdate` .

5. bcrypt e JWT

Crie `gerarHash(senha)` , `conferirSenha(senha, hash)` , `gerarToken(usuario)` (com `id` , `email` e `papel` , expirando em 1 dia) e `lerToken(token)` . Mostre no terminal que o hash muda a cada execução, mas o `compare` continua dando `true` .

6. Autenticar e autorizar

Crie o middleware `autenticar` (401 sem token, com formato errado ou com token inválido) e o middleware `autorizar(...papeis)` (403 se o papel do token não estiver na lista). Proteja `POST` , `PATCH` e `DELETE` de filmes para o papel `admin` .

7. Testes com Vitest e supertest

Escreva testes, com o model mockado, para: `GET /filmes/:id` inexistente (404), `POST /filmes` sem token (401), com token de usuário comum (403) e com token de admin (201), e `PATCH` ignorando um campo proibido.

8. Deploy

Responda: (a) por que o `app.listen` usa `process.env.PORT` ? (b) onde ficam `MONGO_URI` e `JWT_SECRET` no Render? (c) para que serve a rota `/health` ? (d) o que é o cold start do plano free? (e) o que liberar no Atlas para o Render conectar?

Exercícios Extras (Opcional)

9. Swagger

Documente `GET /filmes/:id` no `openapi.js` , com as respostas 200, 400 e 404.

10. Debug

A rota `GET /api/usuarios/perfil` responde 401 mesmo com o token certo. Onde você colocaria breakpoints, e quais variáveis olharia, para descobrir o motivo?

Dica geral: rode `npm test` a cada mudança. Um teste vermelho diz **o que** quebrou; o debugger mostra **por quê**.

LionsDev • Professor Nicolas Cardoso Motta
Lista de Revisão — Preparação para a Avaliação 3